Practical - 10

Aim - Overview of MongoDB

Objective - A NoSQL database: Create and drop-database, collection; data types; insert document; query document; logical operators; update document; delete document; projection; limit records; sort documents.

Pre-Requisites

Before performing MongoDB operations, MongoDB Community Edition and MongoDB Shell (`mongosh`) must be installed on the system.

MongoDB is a NoSQL document-oriented database that stores data in BSON format.

For Windows

Step 1: Download MongoDB

Download MongoDB Community Server from:

MongoDB Official Website -
https://www.mongodb.com/try/download/community?utm_source=chatgpt.com

Step 2: Install MongoDB

1. Open downloaded installer
2. Choose **Complete Setup**
3. Select:
    - Install MongoDB as Service
    - Install MongoDB Compass (optional GUI tool)

4. Complete installation

Step 3: Verify Installation

Open CMD and run:

mongosh

If installed correctly, MongoDB shell will open.

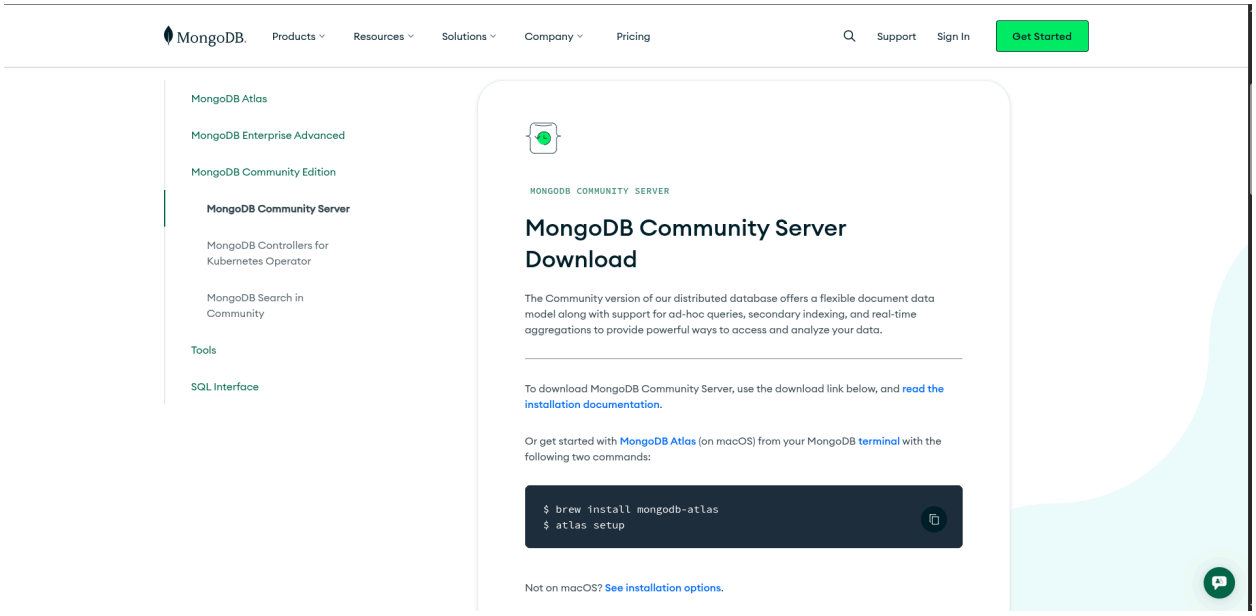
For Linux (Debian/Ubuntu)

PART 1 — Install MongoDB Community Server

Step 1: Open Official Download Page

Open:

MongoDB Community Server Download



Step 2: Select Options

Choose:

Option	Value
Version	Latest Stable Version
Platform	Debian
Package	deb

Then click **Download**.

The downloaded file will look similar to:

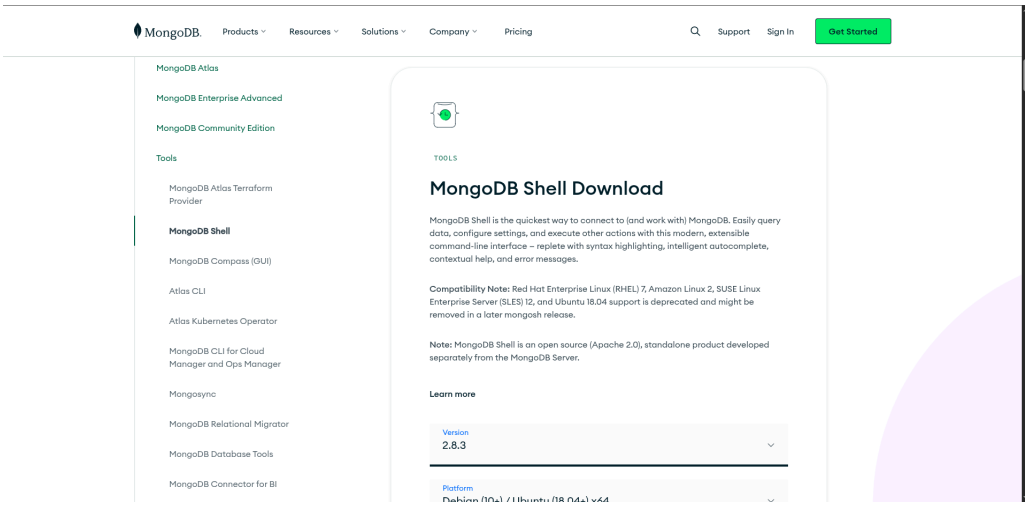
Mongodb-org-server_8.3.1_amd64.deb

PART 2 — Install MongoDB Shell

Step 1: Open Shell Download Page

Open:

[MongoDB Shell Download]



Step 2: Select Options

Choose:

Option	Value
Platform	Debian
Package	deb

Click **Download**.

Downloaded file will look similar to:

mongodb-mongosh_2.8.3_amd64.deb

PART 3 - Installing Packages locally

Step 1: Open Terminal in Downloads Folder

cd ~/Downloads

Step 2: Install MongoDB Server

sudo dpkg -i mongodb-org-server_8.3.1_amd64.deb

```
jsa@workspace:~/Downloads$ sudo dpkg -i mongodb-org-server_8.3.1_amd64.deb
Selecting previously unselected package mongodb-org-server.
(Reading database ... 306899 files and directories currently installed.)
Preparing to unpack mongodb-org-server_8.3.1_amd64.deb ...
Unpacking mongodb-org-server (8.3.1) ...
Setting up mongodb-org-server (8.3.1) ...
Processing triggers for man-db (2.13.1-1) ...
```

Step 3: Fix Dependencies

sudo apt --fix-broken install -y

```
jsa@workspace:~/Downloads$ sudo apt --fix-broken install -y
The following packages were automatically installed and are no longer required:
  liborcus-0.20-0  liborcus-parser-0.20-0  libwoff1  linux-image-6.17.13+deb13-amd64
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 142
```

Step 4: Install MongoDB Shell

sudo dpkg -i mongodb-mongosh_2.8.3_amd64.deb

```
jsa@workspace:~/Downloads$ sudo dpkg -i mongodb-mongosh_2.8.3_amd64.deb
Selecting previously unselected package mongodb-mongosh.
(Reading database ... 306913 files and directories currently installed.)
Preparing to unpack mongodb-mongosh_2.8.3_amd64.deb ...
Unpacking mongodb-mongosh (2.8.3) ...
Setting up mongodb-mongosh (2.8.3) ...
Processing triggers for man-db (2.13.1-1) ...
```

Step 5: Fix Dependencies Again

sudo apt --fix-broken install -y

```
jsa@workspace:~/Downloads$ sudo apt --fix-broken install -y
The following packages were automatically installed and are no longer required:
  liborcus-0.20-0  liborcus-parser-0.20-0  libwoff1  linux-image-6.17.13+deb13-amd64
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 142
```

Step 6: Start MongoDB Service

sudo systemctl start mongod

```
jsa@workspace:~/Downloads$ sudo systemctl enable mongod
Created symlink '/etc/systemd/system/multi-user.target.wants/mongod.service' → '/usr/lib/syste
md/system/mongod.service'.
```

Step 7: Enable MongoDB

sudo systemctl enable mongod

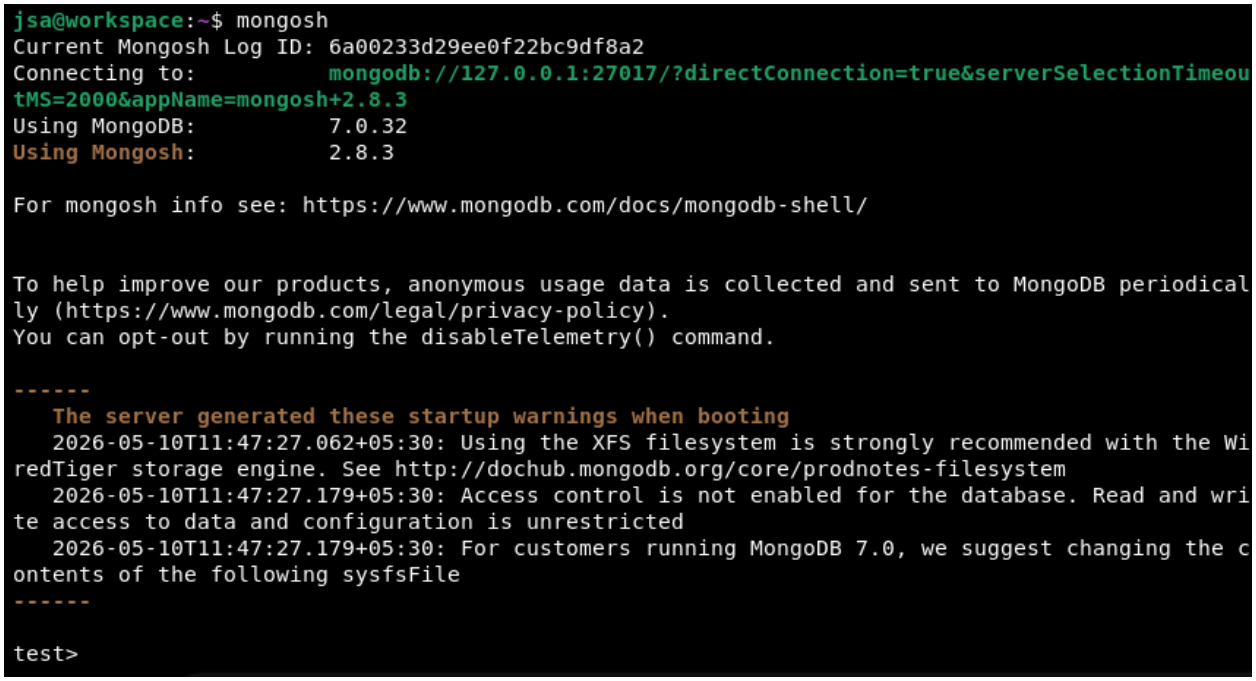
Step 8: Check MongoDB Status

sudo systemctl status mongod

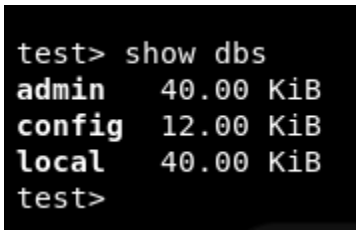
```
jsa@workspace:~/Downloads$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
     Loaded: loaded (/usr/lib/systemd/system/mongod.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-05-10 11:47:26 IST; 3s ago
 Invocation: eae95fe7eb164c838f7cee54a22977c9
       Docs: https://docs.mongodb.org/manual
   Main PID: 17379 (mongod)
     Memory: 82.1M (peak: 82.4M)
        CPU: 188ms
     CGroup: /system.slice/mongod.service
             └─17379 /usr/bin/mongod --config /etc/mongod.conf

May 10 11:47:26 workspace systemd[1]: Started mongod.service - MongoDB Database Server.
May 10 11:47:26 workspace mongod[17379]: {"t":{"$date":"2026-05-10T06:17:26.975Z"},"s":"I",  >
```

Step 9: Open MongoDB Shell

mongosh



Step 10: Test MongoDB



Theory

Introduction to MongoDB

MongoDB is a NoSQL database management system that stores data in the form of collections and documents instead of tables and rows.

MongoDB uses BSON (Binary JSON) format for storing information.

It is widely used in:

- Web applications
- Cloud applications
- Big data systems
- Real-time analytics
- Content management systems

Features of MongoDB

1. Schema-less database
2. High scalability
3. Flexible document structure
4. Fast querying
5. Supports indexing
6. Easy integration with applications

MongoDB Terminology

Relational Database	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	ObjectId

Working on MongoDB

1. **Create Database**

   In MongoDB, databases are created automatically when data is inserted.

   The use command is used to create or switch databases.

```
test> use CollegePortal
switched to db CollegePortal
CollegePortal>
```

2. **View Current Database**

   This command displays the currently selected database.

```
CollegePortal> db
CollegePortal
CollegePortal>
```

3. **Create Collection**

   A Collection in MongoDB stores documents and is similar to a table in relational databases.

```
CollegePortal> db.createCollection("Students")
{ ok: 1 }
CollegePortal>
```

4. **View Collections**

   This command displays all collections present in the database.

```
CollegePortal> show collections
Students
CollegePortal>
```

5. **MongoDB Data Types**

   MongoDB supports multiple data types for storing different kinds of data.

   Common MongoDB Data Types

Data Type	Description
String	Stores text
Integer	Stores numbers
Boolean	True or False
Array	Stores multiple values
Object	Embedded document
Date	Date values

Null	Empty values

Example Document

```
{
    name: "Aman",
    age: 20,
    passed: true,
    subjects: ["DBMS", "OS", "CN"],
    address: {
        city: "Ludhiana",
        state: "Punjab"
    }
}
```

6. Insert Document

Documents are inserted using insertOne() or insertMany() methods.

```
CollegePortal> db.Students.insertMany([
| {
|     name: "Aman",
|     age: 20,
|     course: "B.Tech",
|     marks: 85
| },
| {
|     name: "Simran",
|     age: 21,
|     course: "BCA",
|     marks: 90
| },
| {
|     name: "Rahul",
|     age: 22,
|     course: "MCA",
|     marks: 78
| },
| {
|     name: "Priya",
|     age: 20,
|     course: "B.Tech",
|     marks: 95
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0025b829ee0f22bc9df8a3'),
    '1': ObjectId('6a0025b829ee0f22bc9df8a4'),
    '2': ObjectId('6a0025b829ee0f22bc9df8a5'),
    '3': ObjectId('6a0025b829ee0f22bc9df8a6')
  }
}
```

7. Query Documents

The find() method retrieves documents from a collection.

```
CollegePortal> db.Students.find()
[
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a3'),
    name: 'Aman',
    age: 20,
    course: 'B.Tech',
    marks: 85
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a4'),
    name: 'Simran',
    age: 21,
    course: 'BCA',
    marks: 90
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a5'),
    name: 'Rahul',
    age: 22,
    course: 'MCA',
    marks: 78
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a6'),
    name: 'Priya',
    age: 20,
    course: 'B.Tech',
    marks: 95
  }
]
```

8. Query Specific Documents

This query displays students belonging to the B.Tech course.

```
CollegePortal> db.Students.find({course: "B.Tech"})
[
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a3'),
    name: 'Aman',
    age: 20,
    course: 'B.Tech',
    marks: 85
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a6'),
    name: 'Priya',
    age: 20,
    course: 'B.Tech',
    marks: 95
  }
]
```

9. Logical Operators

Logical operators are used to apply multiple conditions.

Common Operators

Operator	Purpose
$and	All conditions true
$or	Any condition true
$not	Negates condition
$nor	None condition true

```
CollegePortal> db.Students.find({
|     $and: [
|         {course: "B.Tech"},
|         {marks: {$gt: 80}}
|     ]
| })
[
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a3'),
    name: 'Aman',
    age: 20,
    course: 'B.Tech',
    marks: 85
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a6'),
    name: 'Priya',
    age: 20,
    course: 'B.Tech',
    marks: 95
  }
]
```

10. Update Document

Documents are updated using updateOne() or updateMany() methods.

```
CollegePortal> db.Students.updateOne(
|     {name: "Rahul"},
|     {$set: {marks: 82}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

11. Delete Document

Documents are removed using deleteOne() or deleteMany() methods.

```
CollegePortal> db.Students.deleteOne({name: "Simran"})
{ acknowledged: true, deletedCount: 1 }
```

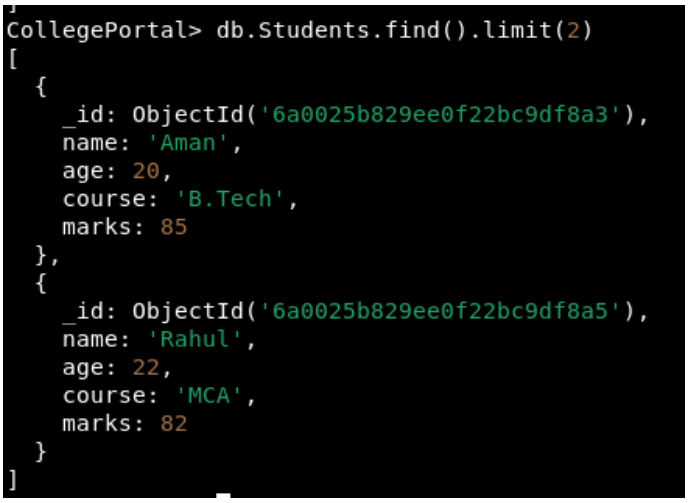
12. Projection

Projection displays selected fields from documents.

```
CollegePortal> db.Students.deleteOne({name: "Simran"})
{ acknowledged: true, deletedCount: 1 }
CollegePortal> db.Students.find(
|     {},
|     {name: 1, marks: 1}
| )
[
  { _id: ObjectId('6a0025b829ee0f22bc9df8a3'), name: 'Aman', marks: 85 },
  { _id: ObjectId('6a0025b829ee0f22bc9df8a5'), name: 'Rahul', marks: 82 },
  { _id: ObjectId('6a0025b829ee0f22bc9df8a6'), name: 'Priya', marks: 95 }
]
```

13. Limit Records

The limit() method restricts the number of documents returned.



14. Sort Documents

The sort() method arranges documents in ascending or descending order.

Sort Values

Value	Meaning
1	Ascending
-1	Descending

```
CollegePortal> db.Students.find().sort({marks: -1})
[
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a6'),
    name: 'Priya',
    age: 20,
    course: 'B.Tech',
    marks: 95
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a3'),
    name: 'Aman',
    age: 20,
    course: 'B.Tech',
    marks: 85
  },
  {
    _id: ObjectId('6a0025b829ee0f22bc9df8a5'),
    name: 'Rahul',
    age: 22,
    course: 'MCA',
    marks: 82
  }
]
```

15. Drop Collection

Collections are removed using the drop() method.

```
CollegePortal> db.Students.drop()
true
```

16. Drop Database

Databases are removed using dropDatabase().

```
CollegePortal> db.dropDatabase()
{ ok: 1, dropped: 'CollegePortal' }
```